

Query Store Triage

#sql-server #performance #diagnostics #query-store

What Query Store Is

Query Store is SQL Server's built-in flight recorder for query performance. Enabled at the database level, it captures every query's text, execution plan(s), and runtime statistics (reads, CPU, duration, row counts) over configurable time windows. It's available in SQL Server 2016 and later, and in Azure SQL.

Think of it as the difference between guessing which queries are slow and knowing.

The Triage Query

This is the query that ranks the worst offenders by total I/O impact:

```
Declare @LookbackHours Int = 24

;With RankedQueries As (
    Select
        qsq.query_id,
        qsp.plan_id,
        qsqt.query_sql_text,
        qsrs.avg_logical_io_reads,
        qsrs.avg_cpu_time,
        qsrs.avg_duration,
        qsrs.count_executions,
        Cast(qsrs.avg_logical_io_reads As Bigint)
            * Cast(qsrs.count_executions As Bigint) As TotalReads,
        qsp.is_forced_plan,
        qsrs.first_execution_time,
        qsrs.last_execution_time
    From sys.query_store_runtime_stats qsrs
    Join sys.query_store_plan qsp
        On qsp.plan_id = qsrs.plan_id
    Join sys.query_store_query qsq
        On qsq.query_id = qsp.query_id
    Join sys.query_store_query_text qsqt
        On qsqt.query_text_id = qsq.query_text_id
    Where qsrs.last_execution_time >= DateAdd(Hour, -@LookbackHours, GetUtcDate())
)
Select Top 50
    query_id,
    plan_id,
    Left(query_sql_text, 200) As QueryPrefix,
    avg_logical_io_reads As AvgReads,
    avg_cpu_time As AvgCpuUs,
    avg_duration As AvgDurationUs,
    count_executions As Execs,
    TotalReads,
    is_forced_plan,
    first_execution_time,
    last_execution_time
From RankedQueries
```

Order By TotalReads Desc

Reading the Results

Key Columns

Column	What It Tells You
TotalReads	AvgReads × Execs. The proc's total I/O footprint. This is the primary ranking metric.
AvgReads	Logical reads per execution. High values mean the query is individually expensive.
Execs	Execution count in the window. High-frequency procs with moderate reads can dominate total I/O.
AvgCpuUs	CPU microseconds. Compare to AvgDuration. If CPU is at or above duration, the query runs serial (possible Scalar UDF Parallelism Barrier). If CPU is above duration, it is parallel.
AvgDuration	Wall clock time in microseconds. What users actually feel.
is_forced_plan	If 1, someone has forced a specific plan via Query Store. Check for forced plan failures.

Triage Priority Matrix

AvgReads	Execs	Priority	Action
High	High	CRITICAL	Fix immediately. Massive total I/O.
High	Low	Important	Fix when able. Individually expensive but infrequent.
Low	High	Monitor	Small per-exec cost but volume matters. Revisit if total grows.
Low	Low	Ignore	Not impacting the system.

Detecting Plan Instability

When the same query_id appears with multiple plan_id values, you have plan instability. This is the hallmark of **Parameter Sniffing**:

Select

```
    qsq.query_id,  
    Count(Distinct qsp.plan_id) As PlanCount,  
    Min(qsrs.avg_logical_io_reads) As BestPlanReads,  
    Max(qsrs.avg_logical_io_reads) As WorstPlanReads,  
    Max(qsrs.avg_logical_io_reads) * 1.0  
      / NullIf(Min(qsrs.avg_logical_io_reads), 0) As ReadRatio  
From sys.query_store_query qsq  
Join sys.query_store_plan qsp On qsp.query_id = qsq.query_id  
Join sys.query_store_runtime_stats qsrs On qsrs.plan_id = qsp.plan_id
```

```
Where qsrs.last_execution_time >= DateAdd(Hour, -24, GetUtcDate())
Group By qsq.query_id
Having Count(Distinct qsp.plan_id) > 1
Order By ReadRatio Desc
```

A ReadRatio of 100x (like the one shown on lsp_RxfGetListOfManualFillGroups, 24K to 2.4M reads across 6 plans) is a clear signal for **Parameter Sniffing** mitigation.

Forced Plan Failures

When a plan is forced but can't be used (schema change, dropped index, statistics update), Query Store records a forced plan failure:

```
Select
    qsp.plan_id,
    qsp.query_id,
    qsp.force_failure_count,
    qsp.last_force_failure_reason_desc
From sys.query_store_plan qsp
Where qsp.is_forced_plan = 1
And qsp.force_failure_count > 0
Order By qsp.force_failure_count Desc
```

In the analysis, lsp_RxfGetListOfManualFillGroups had 91,401 NO_PLAN failures, meaning someone forced a plan that can no longer be reproduced. The optimizer silently falls back to whatever plan it generates, but every fallback is a recompile with whatever parameter values happen to be current. This creates a worst-of-both-worlds scenario: the overhead of plan forcing with none of the stability benefits.

The Systematic Workflow

1. Run the triage query with a 24-hour lookback.
2. Sort by TotalReads to find the biggest I/O consumers.
3. Check for plan instability on the top offenders. Multiple plans suggests parameter sniffing as a candidate.
4. Check for forced plan failures. These are silent performance bombs.
5. Pick the worst offender and analyze the proc code for anti-patterns.
6. Refactor, test with SET STATISTICS IO, verify reads dropped.
7. Re-run triage to confirm the proc dropped in the rankings and find the next target.
8. Repeat. Your score is total system reads.

Real-World Triage Results

From our QueryStoreOutput_20260320.xlsx analysis:

Rank	Procedure	AvgReads	Execs	TotalReads	Issue
1	Ad-hoc query batch	2.2M	7K	15.8B	Unknown, needs investigation
4	lsp_ImgGetListOfTopKImagesToMove	2.5M	8K	8.6B	LEFT JOIN OR Anti-Pattern + UNION ALL Views

Rank	Procedure	AvgReads	Execs	TotalReads	Issue
7	Isp_RxfGetListOfManufacturers	62K Fill Groups	139K	8.7B	9 correlated subqueries (Correlated Subqueries to CTEs) + Scalar UDF Parallelism Barrier
-	Isp_ShpGetOrdersForTopReadyToShipGroup	Top 5	high	high	Plan instability (Parameter Sniffing), 100x read range

Related Concepts

- **Parameter Sniffing**: the number-one cause of plan instability detected by Query Store.
- **Density Vector**: use skew analysis to confirm whether sniffing is the root cause.
- **Scalar UDF Parallelism Barrier**: CPU at or above duration in Query Store signals serial execution.